



Formal Verification of Multi-Agent Systems

Vadim Malvone

vadim.malvone@telecom-paris.fr



(these slides are inspired by tutorials given by Aniello Murano and Wojtek Jamroga)



Outline of the course

- 1. Formal Verification of Strategic Properties**
- 2. Imperfect Information**
- 3. Complexity of Verification**



Formal Verification of Strategic Properties

Introduction

- **Algorithm by Alur, Kupferman, and Henzinger.**
- **Principle:**
 - It is based on a marking algorithm which takes a CGS M , a formula φ of ATL, and consists in associating the states s in M that satisfies φ' , for each sub-formula φ' of φ .
 - Formally, we use the set $[\varphi'] = \{s \in St \mid s \models \varphi'\}$.
 - Finally, we can decide if $s \models \varphi$ by consulting the different markings, i.e. $s \models \varphi$ iff $s \in [\varphi]$.

A more convenient ATL: the Existential Normal Form (ENF)

- ATL is formed according to the grammar:

$$\varphi := p \mid \neg\varphi \mid \varphi \vee \varphi \mid \langle\langle A \rangle\rangle X \varphi \mid [[A]] X \varphi \mid \langle\langle A \rangle\rangle \varphi U \varphi \mid [[A]] \varphi U \varphi$$

where $p \in AP$

- ATL in ENF is formed according to the grammar:

$$\varphi := p \mid \neg\varphi \mid \varphi \vee \varphi \mid \langle\langle A \rangle\rangle X \varphi \mid \langle\langle A \rangle\rangle \varphi U \varphi \mid \langle\langle A \rangle\rangle \varphi R \varphi$$

where $p \in AP$

Equivalence between the two languages

$$[[A]] X \psi = \neg (\llbracket A \rrbracket X \neg \psi)$$

$$[[A]] \psi_1 U \psi_2 = \neg (\llbracket A \rrbracket \neg \psi_1 R \neg \psi_2)$$

The set of subformulas of φ

Function $\text{sub}(\varphi)$

- if $(\varphi = p)$ then return p ;
- if $(\varphi = \neg\psi)$ then return $\text{sub}(\psi) \cup \varphi$;
- if $(\varphi = \psi_1 \vee \psi_2)$ then return $\text{sub}(\psi_1) \cup \text{sub}(\psi_2) \cup \varphi$;
- if $(\varphi = \langle\langle A \rangle\rangle X \psi)$ then return $\text{sub}(\psi) \cup \varphi$;
- if $(\varphi = \langle\langle A \rangle\rangle \psi_1 U \psi_2)$ then return $\text{sub}(\psi_1) \cup \text{sub}(\psi_2) \cup \varphi$;
- if $(\varphi = \langle\langle A \rangle\rangle \psi_1 R \psi_2)$ then return $\text{sub}(\psi_1) \cup \text{sub}(\psi_2) \cup \varphi$;

Complexity: $O(|\varphi|)$

The main procedure

Procedure Labelling (M, φ)

for all $\varphi' \in \text{sub}(\varphi)$

switch(φ')

case p : $[\varphi'] = \{s \in \text{St} \mid p \in \text{Lab}(s)\}$;

case $\neg\psi$: $[\varphi'] = \text{St} \setminus [\psi]$;

case $\psi_1 \vee \psi_2$: $[\varphi'] = [\psi_1] \cup [\psi_2]$;

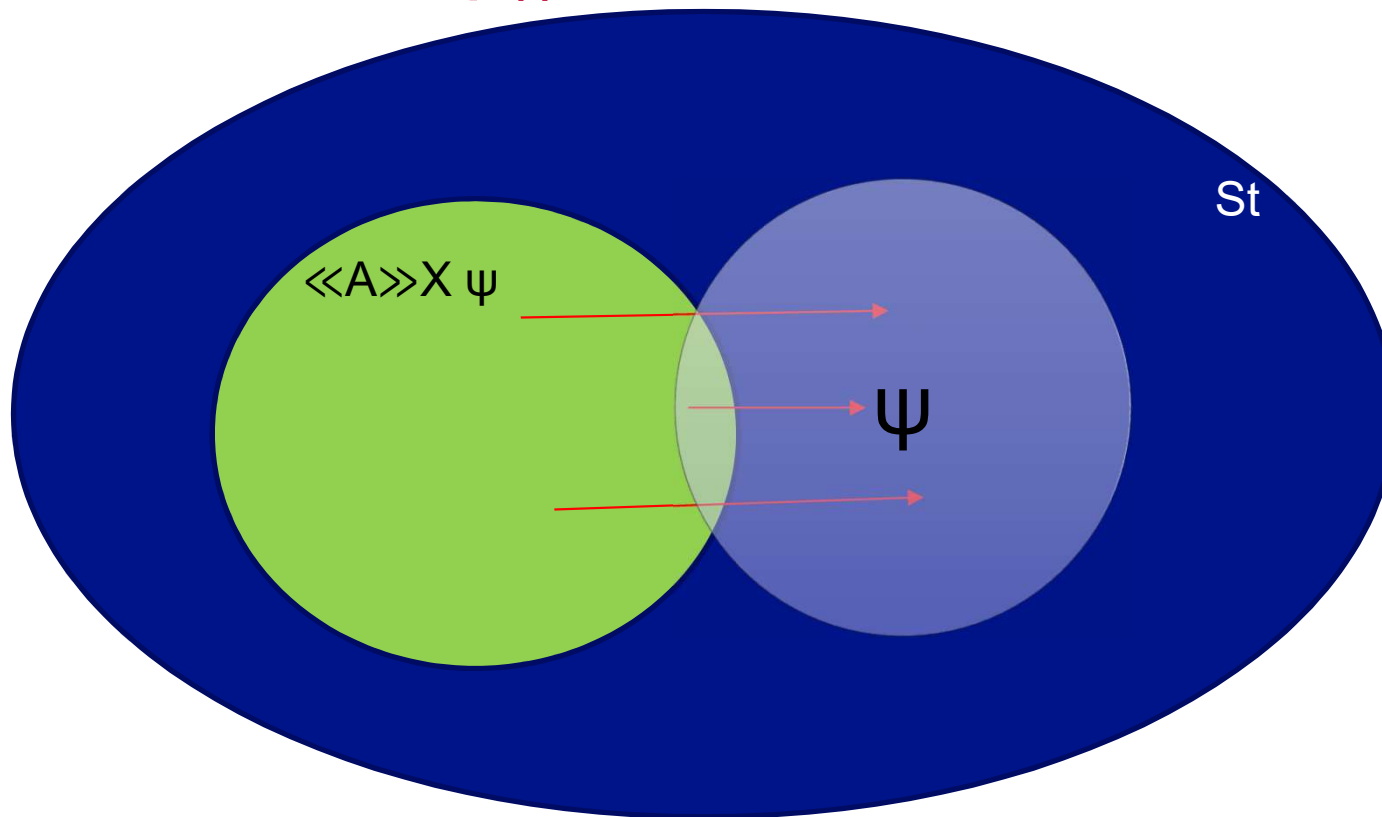
case $\langle\langle A \rangle\rangle X \psi$: $\langle\langle A \rangle\rangle X(M, \varphi')$;

case $\langle\langle A \rangle\rangle \psi_1 U \psi_2$: $\langle\langle A \rangle\rangle U(M, \varphi')$;

case $\langle\langle A \rangle\rangle \psi_1 R \psi_2$: $\langle\langle A \rangle\rangle R(M, \varphi')$;

Complexity: $O(|M|)$

Case $\llbracket A \rrbracket X \psi$ (I)



Case $\ll A \gg X \psi$ (II)

Procedure $\ll A \gg X(M, \varphi)$

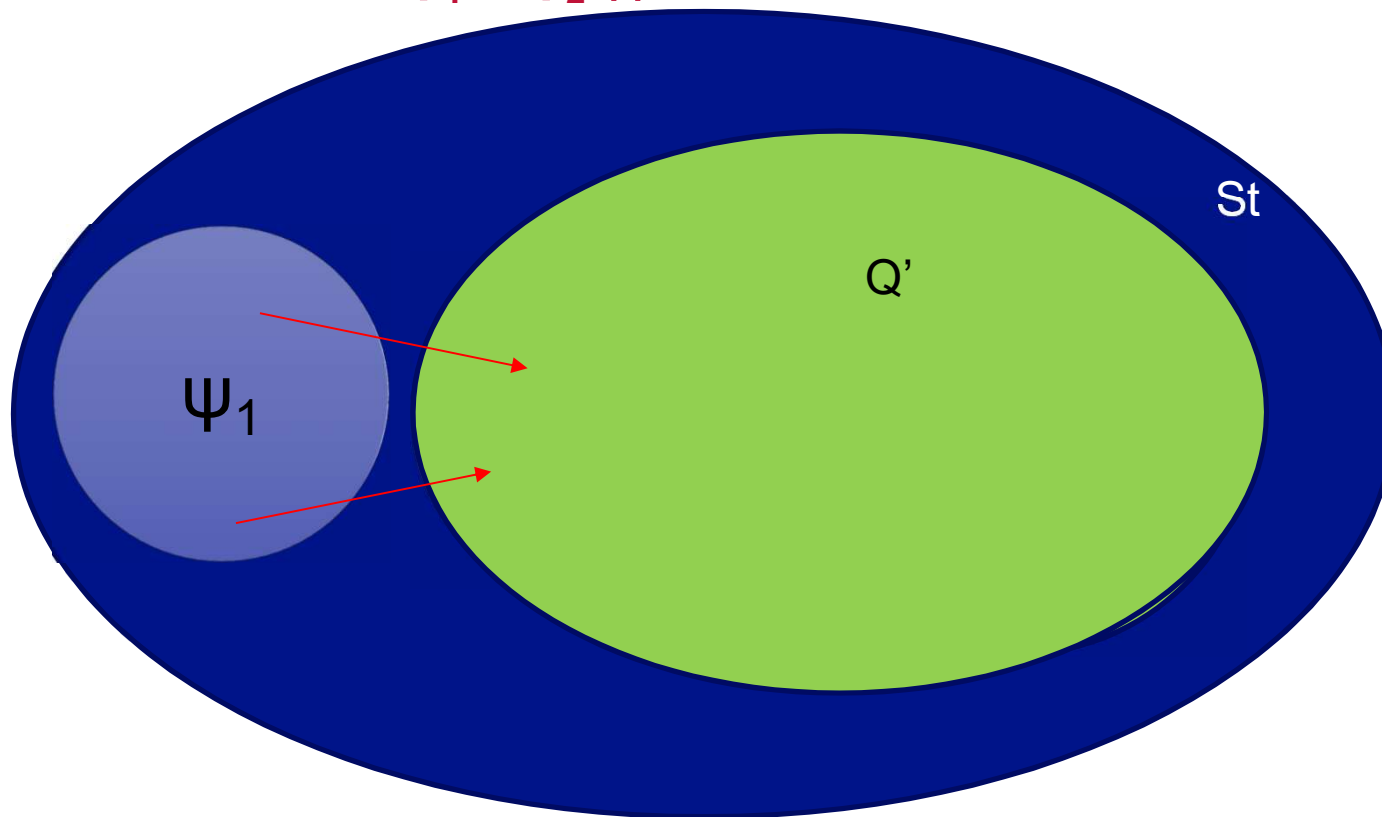
for all $s \in St$

if $(\exists \vec{a}_A \forall \vec{a}_{Ag \setminus A}, T(s, \vec{a}_{Ag}) \in [\psi])$ then

$[\varphi] = [\varphi] \cup \{s\};$

where $\vec{a}_\Gamma$ is a collective action for the coalition Γ , i.e. a tuple of actions, one for each agent in the coalition Γ .

Case $\llbracket A \rrbracket \psi_1 \cup \psi_2 (I)$

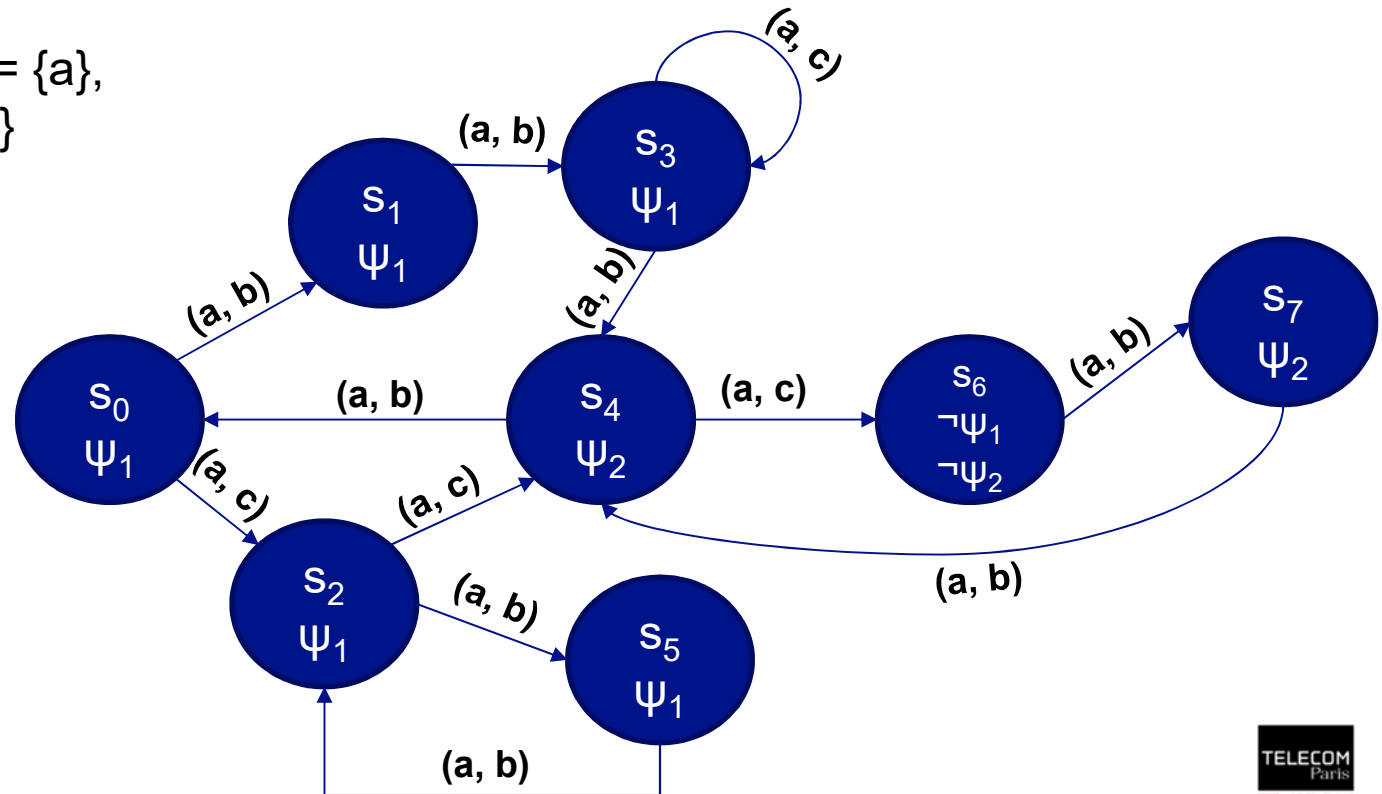


Case $\llbracket A \rrbracket \psi_1 U \psi_2$ (II)

- Collect in a set Q all the states satisfying ψ_2 .
 - All these states also satisfy $\llbracket A \rrbracket \psi_1 U \psi_2$.
- From states in Q , we add in $[\llbracket A \rrbracket \psi_1 U \psi_2]$ all the states t satisfying ψ_1 and from which agents in A can enforce reaching states in $[\llbracket A \rrbracket \psi_1 U \psi_2]$.
- Formally, if $t \in [\psi_1]$ and there exists a collective action $\vec{a}_A$ that for all the collective actions $\vec{a}_{Ag \setminus A}$, $T(t, \vec{a}_{Ag}) \in [\llbracket A \rrbracket \psi_1 U \psi_2]$ holds then $t \in [\llbracket A \rrbracket \psi_1 U \psi_2]$
- Fix point: $\llbracket A \rrbracket \psi_1 U \psi_2 = (\psi_2 \vee (\psi_1 \wedge \llbracket A \rrbracket X \llbracket A \rrbracket \psi_1 U \psi_2))$.

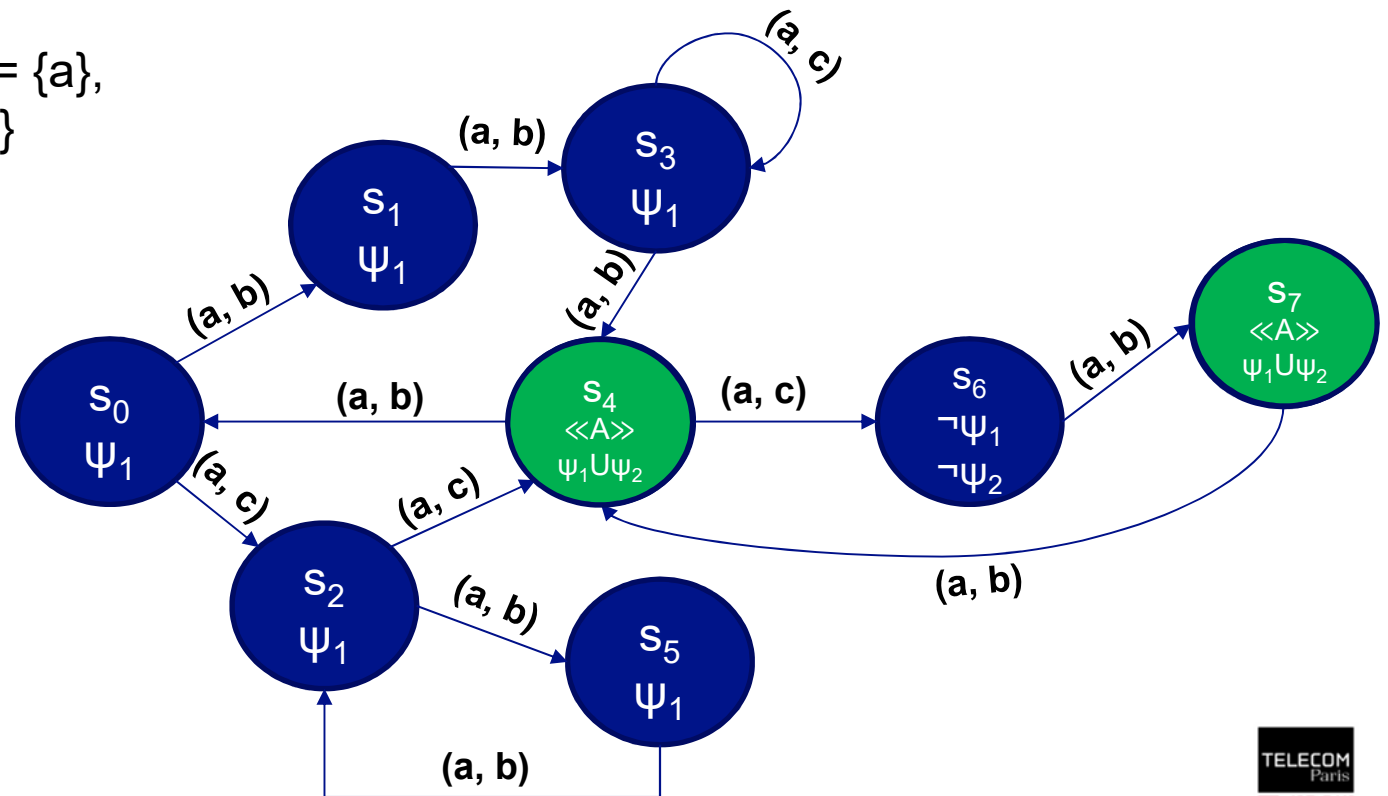
Case $\langle\langle A \rangle\rangle \psi_1 U \psi_2$ (III)

Given $Ag = \{\alpha, \beta\}$, $Ac_\alpha = \{a\}$,
 $Ac_\beta = \{b, c\}$, and $A = \{\beta\}$



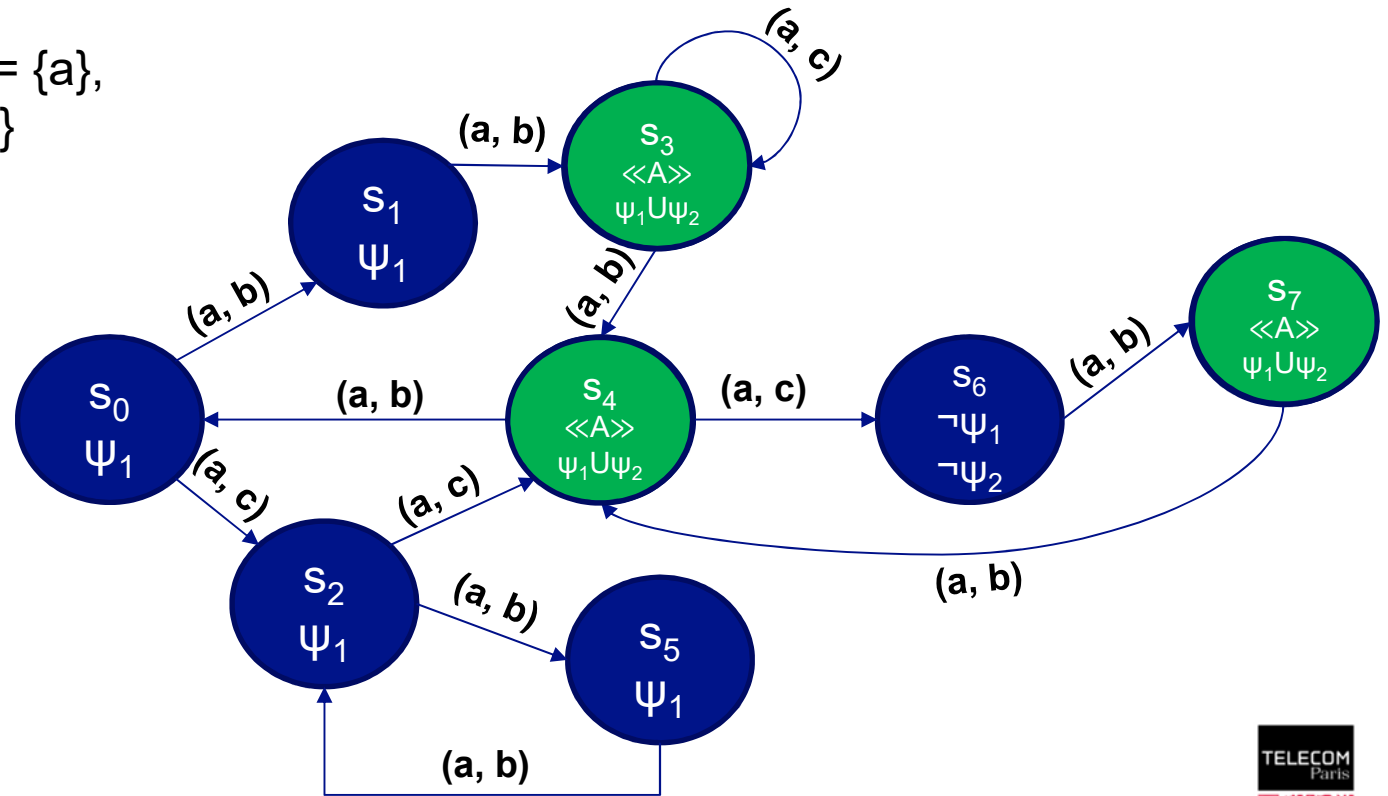
Case $\llbracket A \rrbracket \psi_1 U \psi_2$ (III)

Given $Ag = \{\alpha, \beta\}$, $Ac_\alpha = \{a\}$,
 $Ac_\beta = \{b, c\}$, and $A = \{\beta\}$



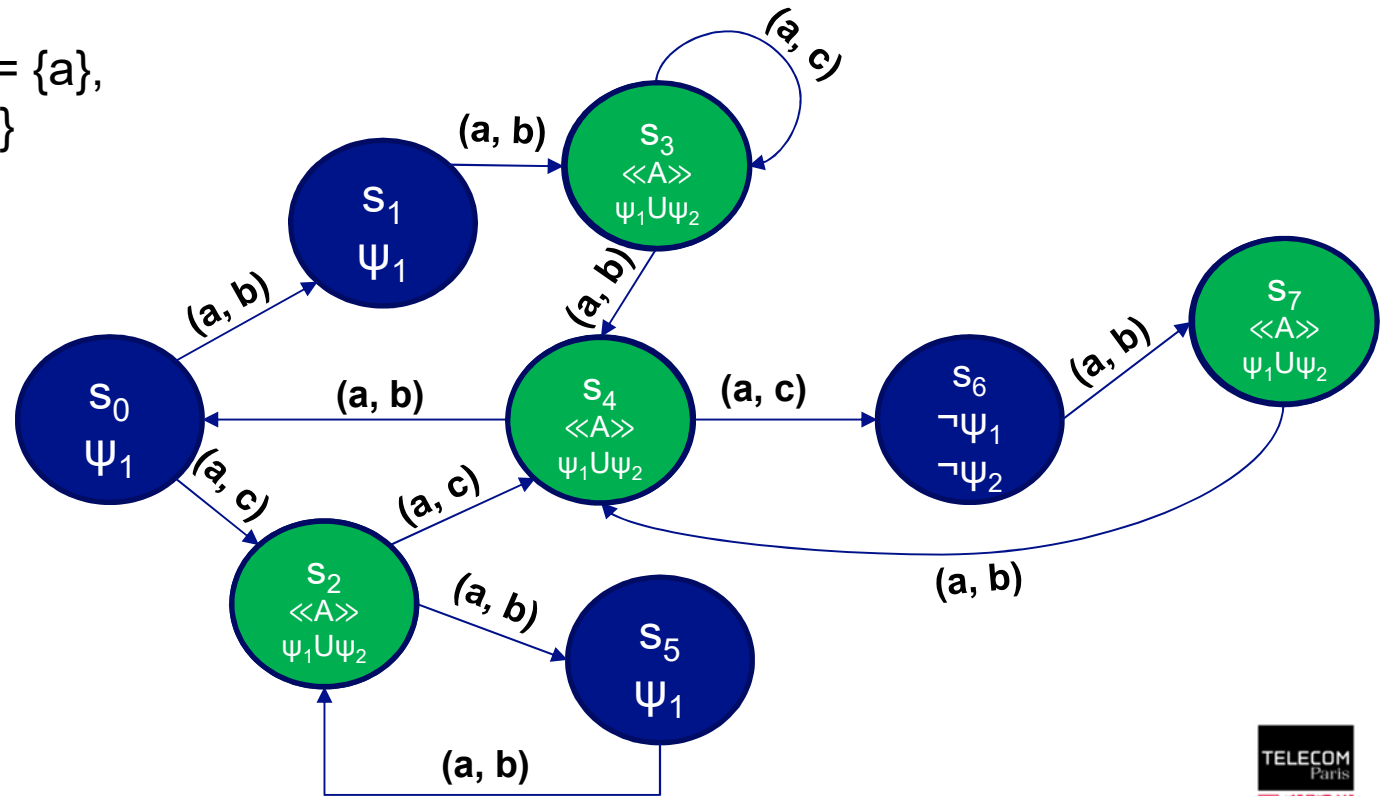
Case $\llbracket A \rrbracket \psi_1 U \psi_2$ (III)

Given $Ag = \{\alpha, \beta\}$, $Ac_\alpha = \{a\}$,
 $Ac_\beta = \{b, c\}$, and $A = \{\beta\}$



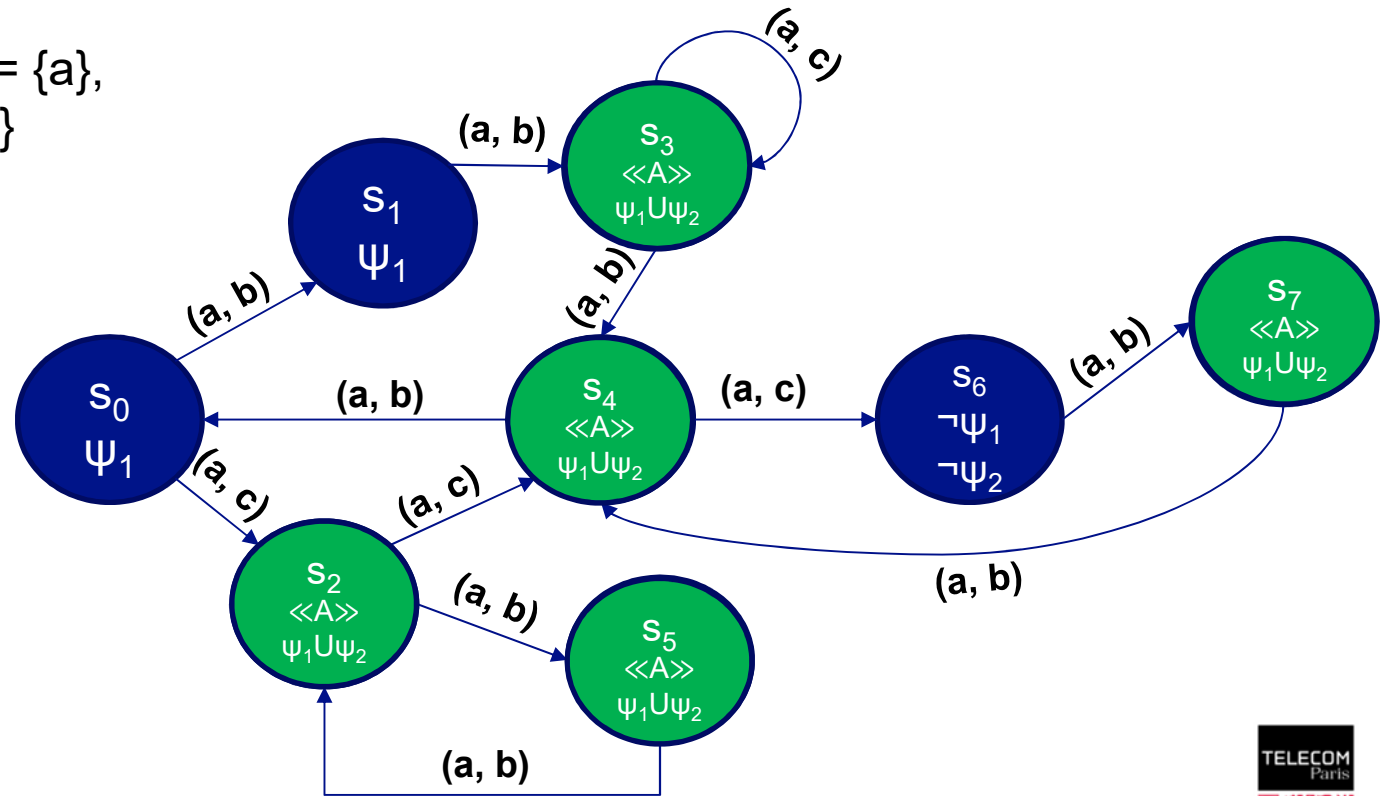
Case $\langle\langle A \rangle\rangle \psi_1 U \psi_2$ (III)

Given $Ag = \{\alpha, \beta\}$, $Ac_\alpha = \{a\}$,
 $Ac_\beta = \{b, c\}$, and $A = \{\beta\}$



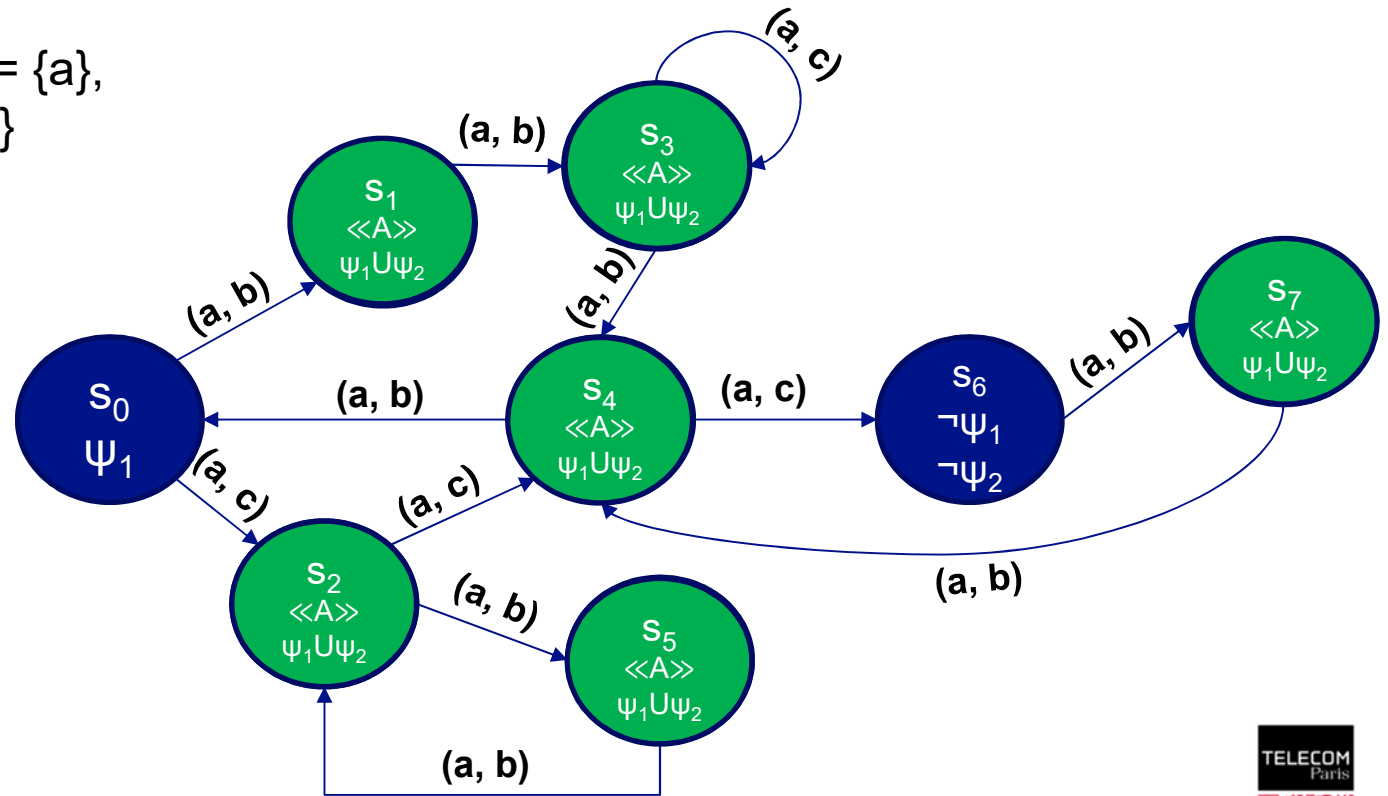
Case $\langle\langle A \rangle\rangle \psi_1 U \psi_2$ (III)

Given $Ag = \{\alpha, \beta\}$, $Ac_\alpha = \{a\}$,
 $Ac_\beta = \{b, c\}$, and $A = \{\beta\}$



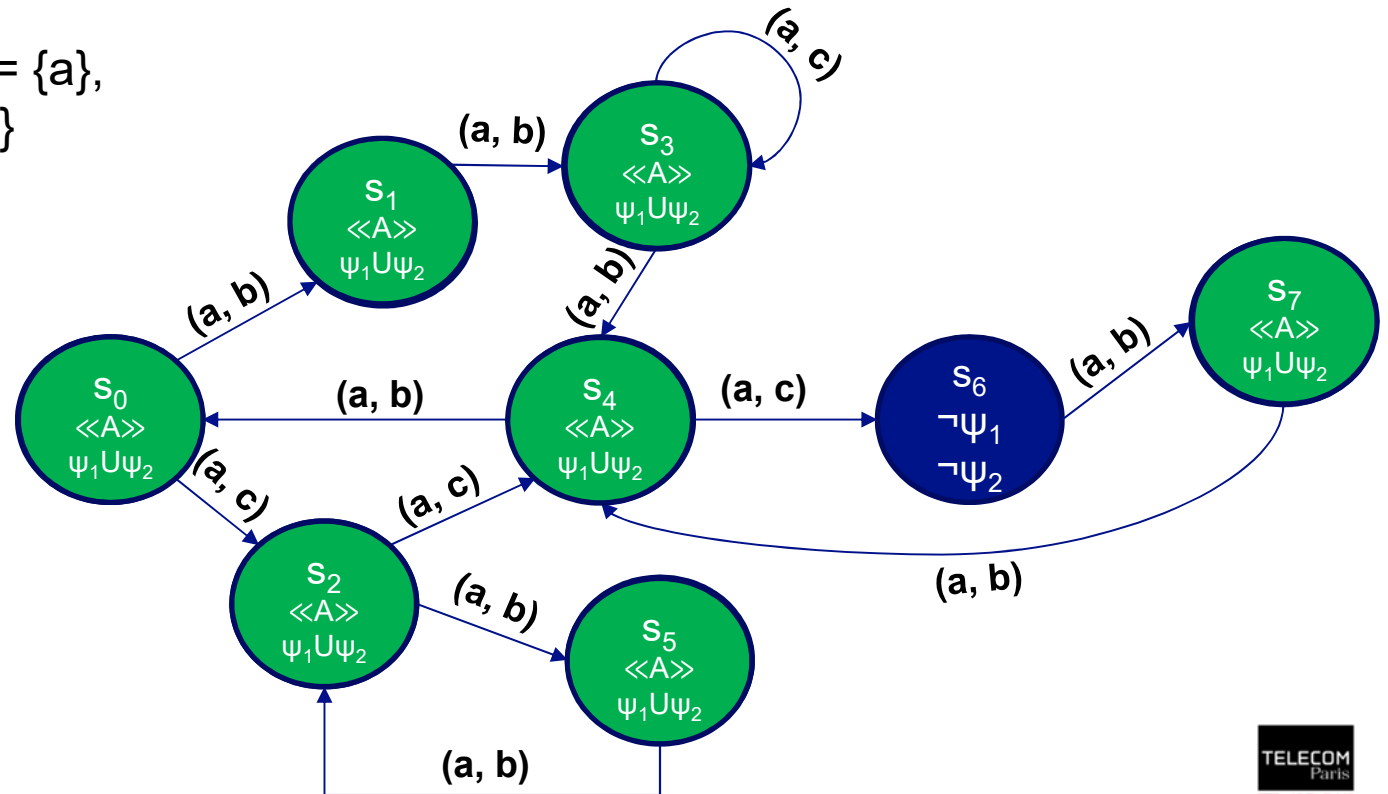
Case $\langle\langle A \rangle\rangle \psi_1 U \psi_2$ (III)

Given $Ag = \{\alpha, \beta\}$, $Ac_\alpha = \{a\}$,
 $Ac_\beta = \{b, c\}$, and $A = \{\beta\}$



Case $\langle\langle A \rangle\rangle \psi_1 U \psi_2$ (III)

Given $Ag = \{\alpha, \beta\}$, $Ac_\alpha = \{a\}$,
 $Ac_\beta = \{b, c\}$, and $A = \{\beta\}$



Case $\ll A \gg \psi_1 U \psi_2$ (IV)

Procedure $\ll A \gg U(M, \varphi)$

for all $s \in St$

if ($s \in [\psi_2]$) then

$[\varphi] = [\varphi] \cup \{s\};$

$Q = \Phi;$

while $Q \neq [\varphi]$

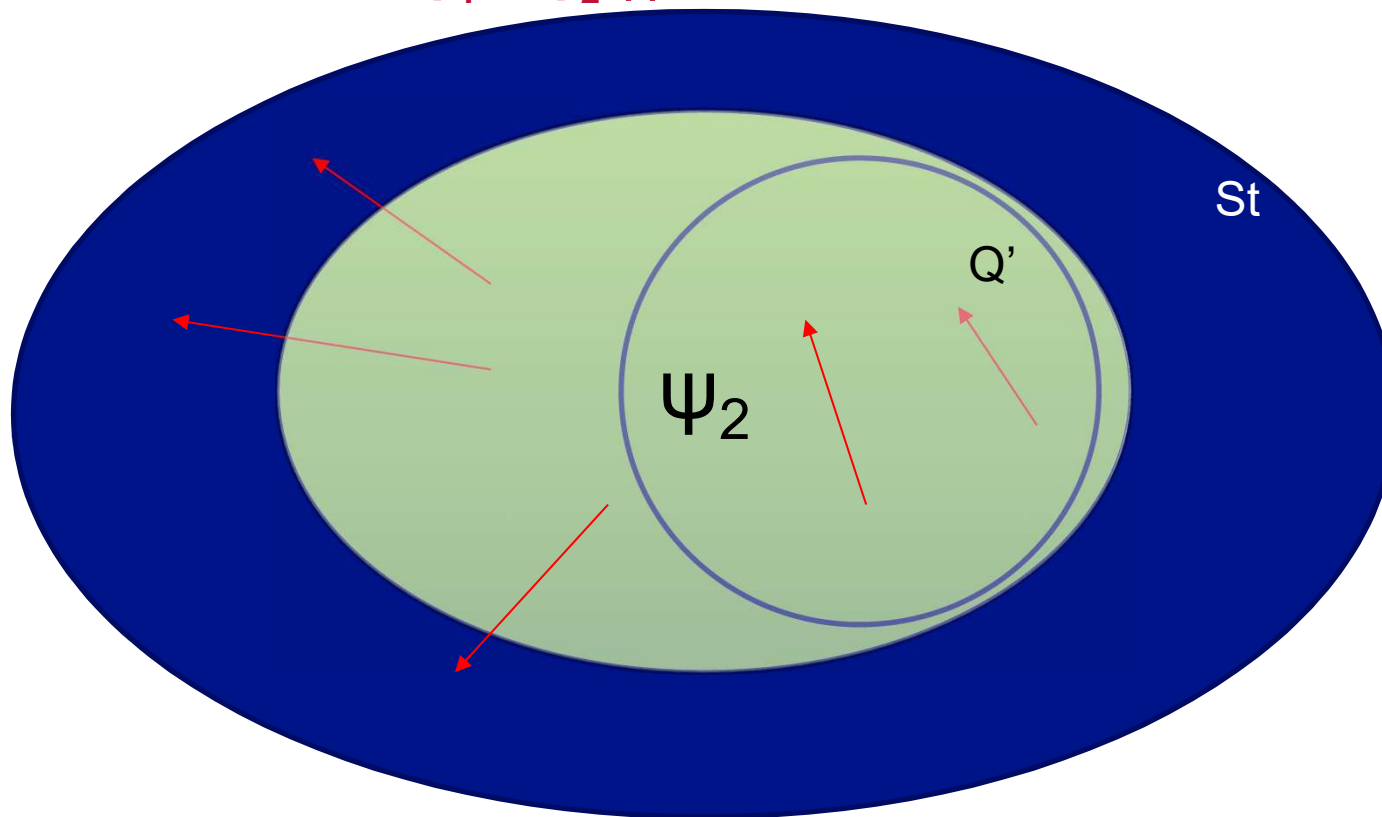
$Q = [\varphi];$

for all $s' \in St$

if ($\exists \vec{a}_A \forall \vec{a}_{Ag \setminus A}, \top(s', \vec{a}_{Ag}) \in [\varphi]$) and ($s' \in [\psi_1]$) and ($s' \notin [\varphi]$) then

$[\varphi] = [\varphi] \cup \{s'\};$

Case $\llbracket A \rrbracket \psi_1 R \psi_2 (I)$

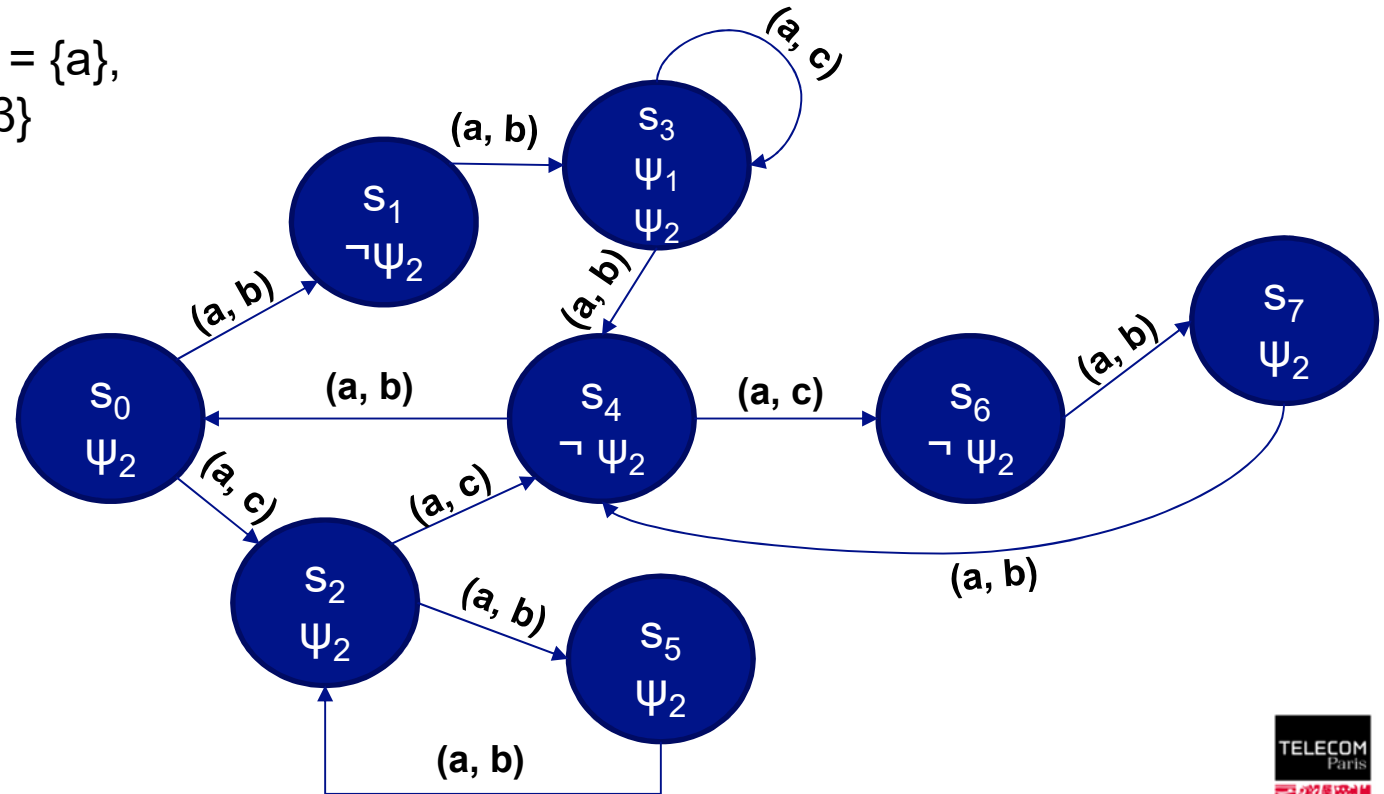


Case $\langle\langle A \rangle\rangle \psi_1 R \psi_2$ (II)

- Collect in a set Q all the states satisfying ψ_2 .
 - All these states represent the largest set of candidates that can satisfy $\langle\langle A \rangle\rangle \psi_1 R \psi_2$.
- From states in Q , we remove each state t that does not satisfy ψ_1 and from t agents in A cannot enforce reaching any state in Q .
- Formally, if $t \in Q$, $t \notin [\psi_1]$, and there is not a collective action $\vec{a}_A$ that for all the collective actions $\vec{a}_{Ag \setminus A}$, $T(t, \vec{a}_{Ag}) \in Q$ holds, then we remove t from Q .
- Fix point: $\langle\langle A \rangle\rangle \psi_1 R \psi_2 = (\psi_2 \wedge (\psi_1 \vee \langle\langle A \rangle\rangle X \langle\langle A \rangle\rangle \psi_1 R \psi_2))$.

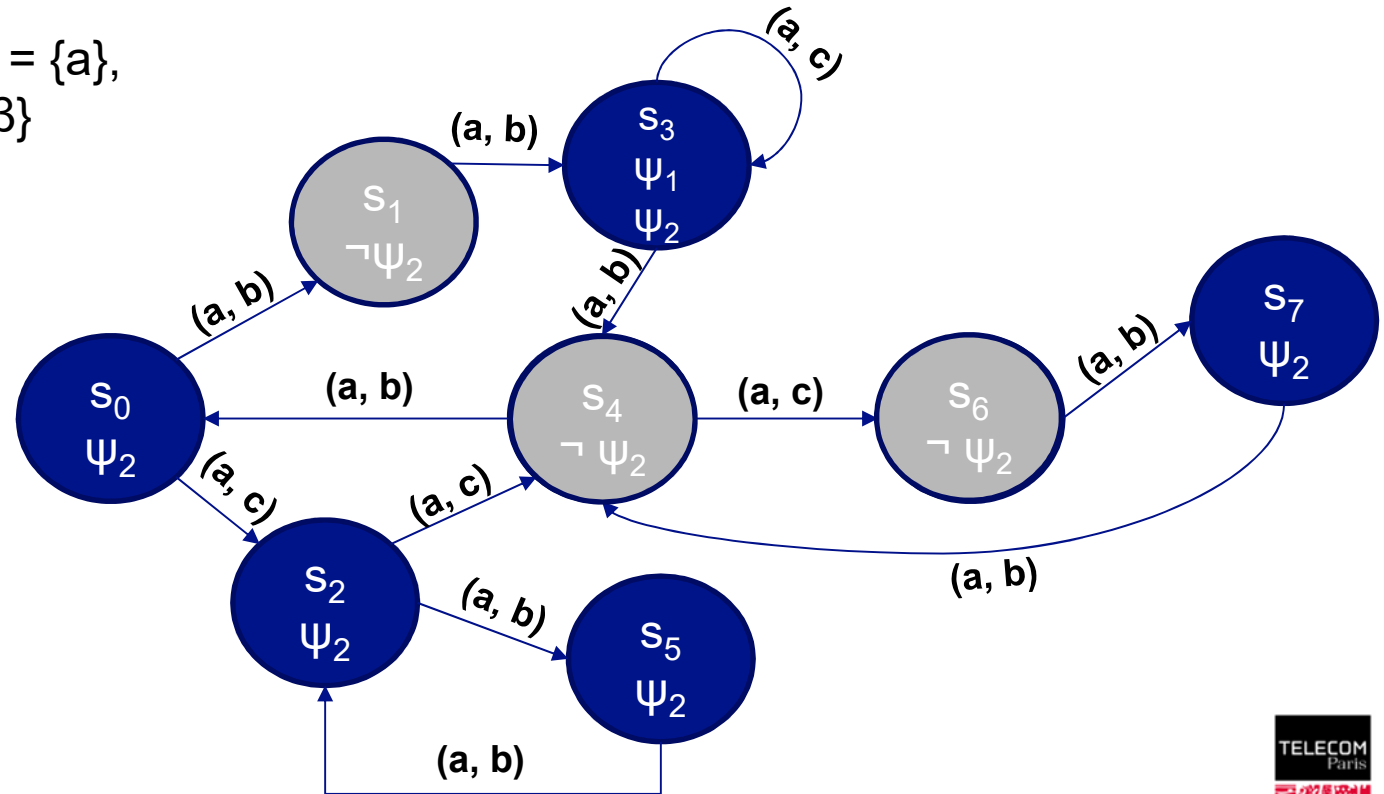
Case $\ll A \gg \psi_1 R \psi_2$ (III)

Given $Ag = \{\alpha, \beta\}$, $Ac_\alpha = \{a\}$,
 $Ac_\beta = \{b, c\}$, and $A = \{\beta\}$



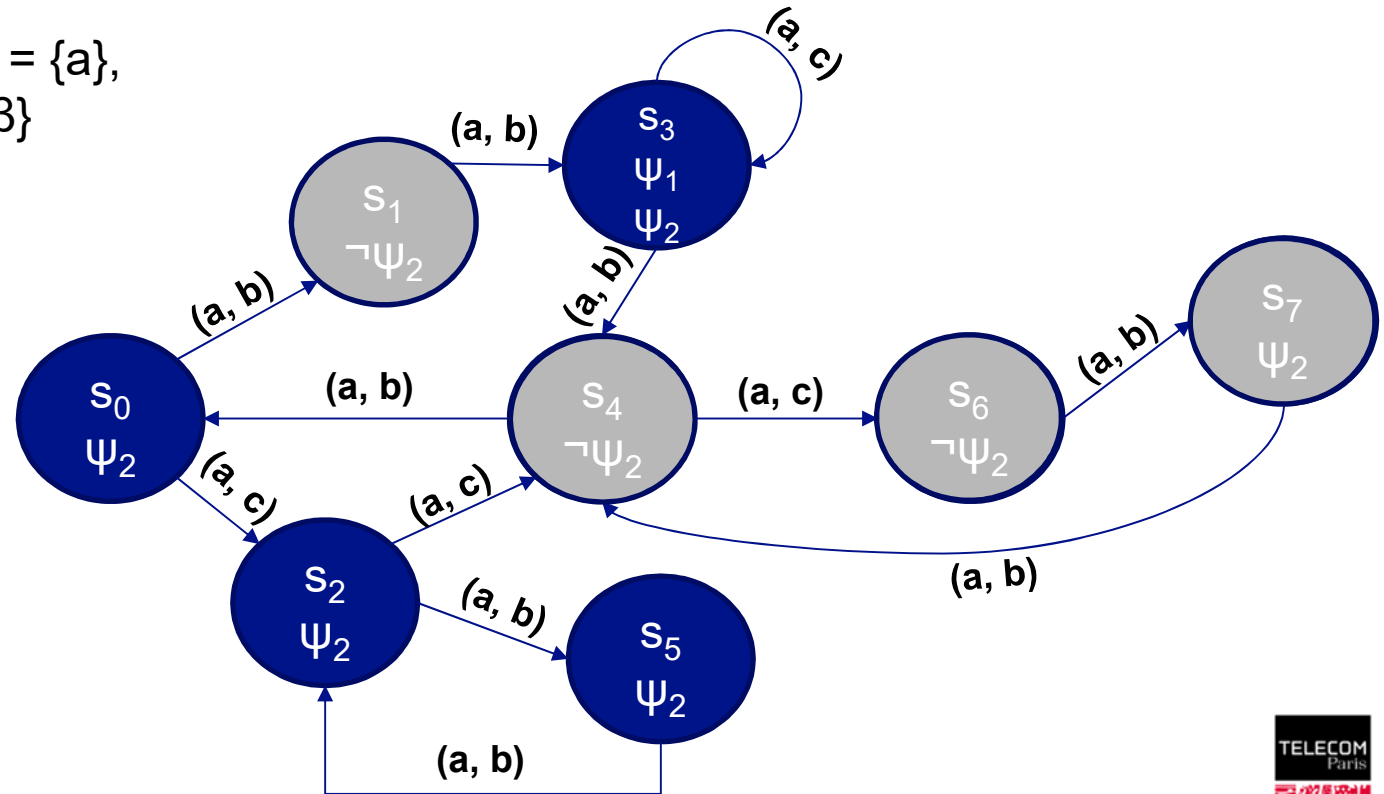
Case $\langle\langle A \rangle\rangle \psi_1 R \psi_2$ (III)

Given $Ag = \{\alpha, \beta\}$, $Ac_\alpha = \{a\}$,
 $Ac_\beta = \{b, c\}$, and $A = \{\beta\}$



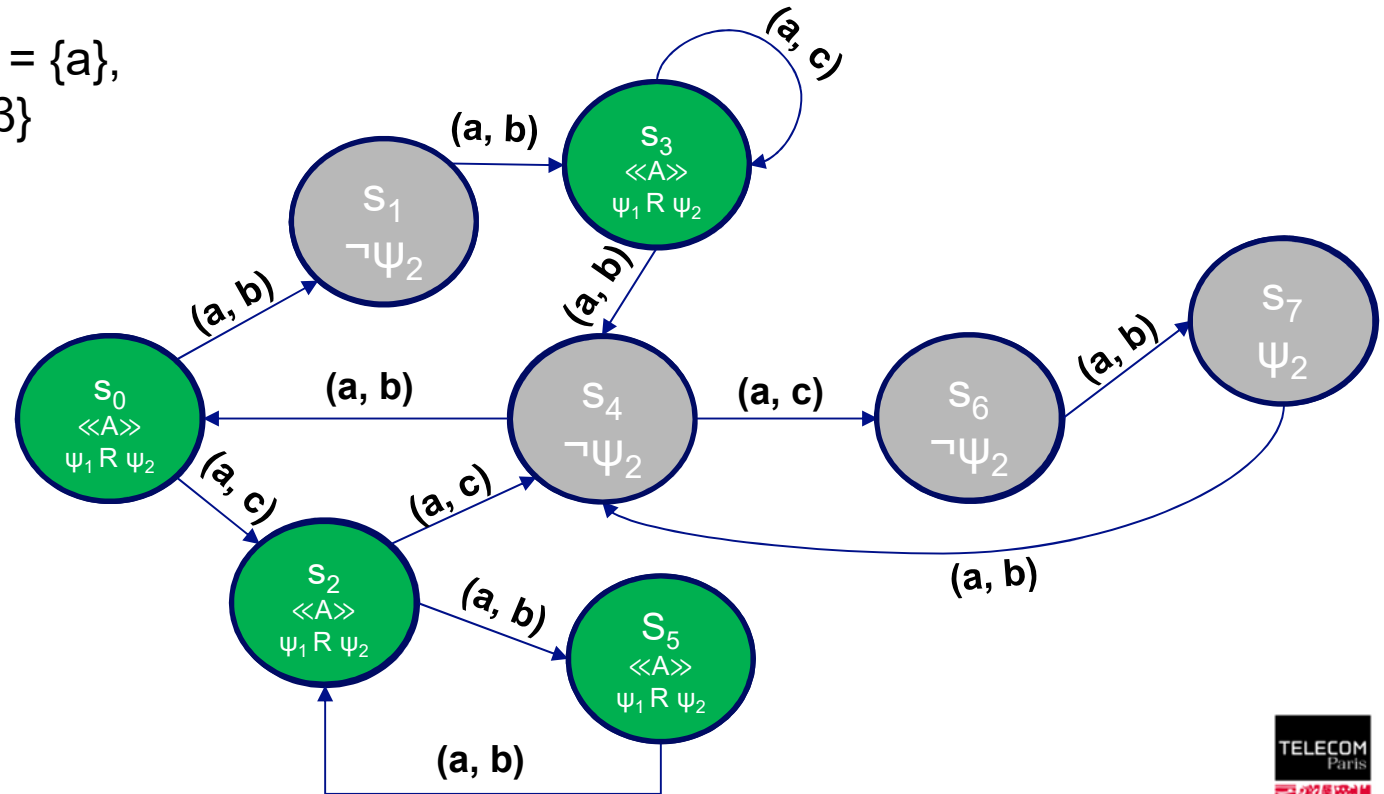
Case $\langle\langle A \rangle\rangle \psi_1 R \psi_2$ (III)

Given $Ag = \{\alpha, \beta\}$, $Ac_\alpha = \{a\}$,
 $Ac_\beta = \{b, c\}$, and $A = \{\beta\}$



Case $\langle\langle A \rangle\rangle \psi_1 R \psi_2$ (III)

Given $Ag = \{\alpha, \beta\}$, $Ac_\alpha = \{a\}$,
 $Ac_\beta = \{b, c\}$, and $A = \{\beta\}$



Case $\ll A \gg \psi_1 R \psi_2$ (IV)

Procedure $\ll A \gg R(M, \varphi)$

$Q_1 = \Phi; Q_2 = [\psi_2];$

while $Q_1 \neq Q_2$

$Q_1 = Q_2; Q_2 = \Phi;$

for all $s' \in Q_1$

if $(\exists \vec{a}_A \forall \vec{a}_{Ag \setminus A}, T(s', \vec{a}_{Ag}) \in Q_1)$ or $(s' \in [\psi_1])$ then

$Q_2 = Q_2 \cup \{s'\};$

$[\varphi] = Q_1;$

ATL model checking complexity

The algorithm presented shows that the model checking problem for ATL can be solved in linear time in the size of the model M and the size of the property φ , that is:

in time $O(|M| \cdot |\varphi|)$

where $|M|$ is the size of the model M and $|\varphi|$ is the number of sub-formulas of φ .



Imperfect Information

Uncertainty

- ATL includes no notion of uncertainty (or, dually of knowledge).
- This makes reasoning in ATL rather unrealistic for MAS.
- Here, we show how to introduce knowledge and uncertainty into reasoning about strategic abilities.
- How can we reason about multi-step games with **imperfect information**?

We extend CGSs with indistinguishability relations.

Model for MAS with imperfect information (I)

A Concurrent Game Structures with imperfect information (iCGS) is a tuple:

$$M = (AP, Ag, St, St_0, Ac, D, T, Lab, \sim)$$

- AP is a set of atomic propositions.
- $Ag = \{1, \dots, |Ag|\}$ is a set of agents.
- St is a set of states.
- $St_0 \subseteq St$ is a set of initial states.
- $Ac = Ac_1 \cup \dots \cup Ac_{|Ag|}$ is a set of actions.
- $D: Ag \times St \rightarrow 2^{Ac}$ defines the actions available to an agent in a state.
- $T: St \times Ac_1 \times \dots \times Ac_{|Ag|} \rightarrow St$ is a transition function.
- $Lab: St \rightarrow 2^{AP}$ labels each state with propositions true in the state.
- $\sim = \{\sim_1, \dots, \sim_{|Ag|}\}$ is a set of equivalence relations over St .

Model for MAS with imperfect information (II)

- Given an iCGS, the notation $s \sim_{\alpha} s'$ means that the states s and s' are indistinguishable for agent α .
- When two states s and s' are indistinguishable for agent α then the actions available in these states are the same, i.e.

$$D(\alpha, s) = D(\alpha, s')$$

- Are these constraints enough? **NO!**
- A notion on strategies needs to be defined.

Strategies under Uncertainty (I)

- $\ll A \gg \varphi$ means A can enforce φ .
- It should at least mean that A is able to execute the right strategy!
- Executable strategies = **uniform strategies**.
- In many cases, we also mean that A is able to identify the strategy.
- In order to identify a strategy as successful, the agents must check their outcome paths from indistinguishable states.

Strategies under Uncertainty (II)

- Strategy σ_α is uniform if and only if it specifies the same choices for indistinguishable situations.
 - Memoryless: if $s \sim_\alpha s'$ then $\sigma_\alpha(s) = \sigma_\alpha(s')$
 - Memoryfull: if $h \sim_\alpha h'$ then $\sigma_\alpha(h) = \sigma_\alpha(h')$

where $h, h' \in \text{St}^+$ and $h \sim_\alpha h'$ iff $h_j \sim_\alpha h'_j$ for every $j > 0$.



Complexity of Verification

Logical Problems (I)

- **Decision problem:** given the representation of an instance, decide whether it belongs to the set of “good” instances.
- **Typical logical problems:** validity, satisfiability, and model checking.
- **Validity:** given formula φ , determine if φ is valid (true in every model).
- **Satisfiability:** given formula φ , determine if φ is satisfiable (true in some model).
- **Model checking:** given formula φ and model M , determine if φ is true in M .



Logical Problems (II)

- **Decision problem can be seen as a Yes/No question.**
 - The answer depends on the input, i.e., the actual parameters.
- **Algorithmic view: we want a machine to answer the question.**
 - We give the input, the machine (algorithm) returns the answer!
- **Will that work?**
 - It depends on how difficult the question is → Computational complexity.

Some Complexity Classes

- **P (polynomial time):** problems solvable in polynomial time in the size of the input.
- **NP (polynomial nondeterministic time):** problems verifiable in polynomial time in the size of the input.
- **PH (polynomial hierarchy):** problems solvable by queries to a multilevel oracle.
- **PSPACE (polynomial space):** problems solvable by using an amount of memory that is polynomial in the size of the input.
- **EXPTIME (exponential time):** problems solvable in exponential time in the size of the input.

Complexity results: Temporal Logics

Temporal logic	Model checking complexity
LTL	PSPACE
CTL	P
CTL*	PSPACE

Complexity results: ATL

Model checking	Perfect information	Imperfect information
Memoryless	P	P ^{NP}
Memoryfull	P	undecidable

Complexity results: ATL*

Model checking	Perfect information	Imperfect information
Memoryless	PSPACE	PSPACE
Memoryfull	2EXPTIME	undecidable